Tuple Space in Erlang

Selssabil Medghaghet
Matricola N.337435

1 Introduction

The purpose of this project was to implement a Tuple Space in Erlang using the Linda model. A Tuple Space is a shared memory abstraction where processes communicate by inserting and retrieving tuples instead of sending direct messages. This makes coordination more decoupled than direct message passing.

The system is split into two modules:

- `ts`: implements the Tuple Space server
- `ts_bench`: handles benchmarking

The Tuple Space runs as a globally registered `gen_server`, allowing all nodes in the cluster to access it by name:

```
gen_server:start_link({global, Name}, ?MODULE, [Name], [])
```

2 Design Decisions

2.1 gen_server

`gen_server` was used because it serializes access to shared state automatically, preventing race conditions without explicit locking. All Tuple Space state are encapsulated inside a single process.

Blocking operations also fit naturally into the OTP model. A `gen_server:call` blocks until a reply is received, which matches Linda semantics for `in` and `rd`.

Implementing the same behavior with a raw process would significantly increase complexity and error risk.

2.2 ETS for storage

Tuples are stored in an ETS table of type `bag`. ETS was chosen instead of a list in the server state because lookup and deletion are approximately $O(1)$, while list scanning is $O(n)$. With 10,000 tuples in the benchmark, this difference becomes significant.

The `bag` type is required because multiple identical tuples must coexist. A `set` would overwrite duplicates.

ETS also supports pattern matching with wildcards:

```
Table = ets:new(Name, [bag, public]),
ets:match_object(Table, {10, '_'})
```

2.3 Blocking and wake-up logic

When `in` or `rd` finds no matching tuple, the caller is stored in a waiting list and the operation blocks:

```
handle_call({in, Pattern}, From, State) ->
    case ets:match_object(State#state.table, Pattern) of
        [T | _] ->
            ets:delete_object(State#state.table, T),
            {reply, {ok, T}, State};
        [] ->
            {Pid, _} = From,
            Ref = erlang:monitor(process, Pid),
            {noreply, State#state{
                waiting = [{Pattern, From, in, Ref}
                           | State#state.waiting]
            }}
    end;
```

The `rd` operation is identical except that the tuple is not removed from ETS.

When a new tuple arrives via `out`, the `wake_waiters` function scans the waiting list.

- Multiple `rd` callers may receive the same tuple
- Only one `in` caller may consume a tuple

A `Consumed` flag prevents multiple deliveries to `in` operations:

```
in when Consumed =:= false ->
    ets:delete_object(Table, Tuple),
    gen_server:reply(From, {ok, Tuple}),
    {Acc, true};
```

Since new waiters are prepended and `lists:foldl/3` traverses left to right, the processing order is LIFO. This affects only the wake-up order, not correctness.

Each waiting process is monitored using `erlang:monitor`. If a process dies or times out, the corresponding entry is removed:

```
handle_info({'DOWN', Ref, process, _Pid, _Reason}, State) ->
    NewWaiting =
        lists:filter(
            fun({_, _, _, R}) -> R =/= Ref end,
            State#state.waiting),
    {noreply, State#state{waiting = NewWaiting}};
```

2.4 Timeouts

For `in/3` and `rd/3`, the timeout is handled by `gen_server:call`. If it expires, the call raises an exit signal that is caught by the client:

```
in(TS, Pattern, Timeout) ->
    try gen_server:call({global, TS}, {in, Pattern}, Timeout) of
        Res -> Res
    catch
        exit:{timeout, _} -> {err, timeout}
    end.
```

When a timeout occurs, the monitor ensures the waiting entry is eventually cleaned up via a `DOWN` message.

2.5 Pattern matching

Patterns use '_' as wildcard syntax.

ETS performs matching directly for stored tuples. For waiting processes, a custom function is used:

```
matches(Pattern, Tuple) ->
    try
        lists:all(
            fun({P, T}) ->
                P =:= '_' orelse P =:= T
            end,
            lists:zip(
                tuple_to_list(Pattern),
                tuple_to_list(Tuple)
            )
        )
    catch
        _:_ -> false
    end.
```

The `try/catch` prevents malformed inputs from crashing the server.

2.6 Distribution

The Tuple Space is globally registered using {`global, Name`}, allowing any node to access it without knowing its physical location.

A centralized architecture was chosen, where a single node manages all state. A replicated design was not implemented due to the need for consensus and synchronization mechanisms.

Node failures are handled using `erlang:monitor_node/2`, allowing the system to receive `nodedown` events without terminating.

```
erlang:monitor_node(Node, true)
```

When a node disconnects,the internal state is updated safely:

```
handle_info({nodedown, Node}, State) ->
    NewNodes = lists:delete(Node, State#state.nodes),
    {noreply, State#state{nodes = NewNodes}};
```

3 Measurements

All benchmarks used $N = 10{,}000$ operations measured using:

```
erlang:monotonic_time(microsecond)
```

A synchronization barrier using:

```
ts:rd(bench_ts, {N, '_'})
```

ensures that all previously issued asynchronous `out` operations have been processed before the timer stops. Since Erlang preserves message ordering between processes, observing the final tuple implies that all earlier tuples were already inserted.

The recovery benchmark used two Erlang nodes on the same machine with `net_ticktime` set to 2 seconds to measure failure detection time.

3.1 Benchmark Results

Operation	Average latency	Notes
out/2	∼3 μs	async cast, no reply needed
rd/2	∼2–3 μs	sync call + ETS lookup
in/2	∼2–3 μs	ETS lookup + deletion
node recovery	∼6–7 s	`net_ticktime = 2s`

3.2 Analysis

`out` measures end-to-end latency from sending the cast until the tuple becomes visible in ETS. Because it is asynchronous, measuring only dispatch time would underestimate the actual execution cost.

`rd` and `in` are synchronous operations, so each call already includes completion time. Their performance is similar because the additional ETS deletion in `in` is negligible compared to message passing overhead.

The wake-up mechanism had no measurable impact during benchmarking since no processes were blocked.

Node recovery latency is dominated by BEAM's tick-based node failure detection. With net_ticktime = 2, failure detection may take up to approximately 8 seconds in the worst case.